



Bridgistic

Bridgistic

Complete User Guide

Give Claude and Claude Cowork production-safe, scoped control of WordPress — signed requests, approvals, one-call rollback, and scheduled automation.

[For beginners & pros](#)[Features · Setup · Best practice](#)[WordPress 6.4+ · PHP 8.0+](#)

By **Shuvo Sarker** · WordPressistic • June 2026 • github.com/Shubochandrosarker/bridgistic

CONTENTS

What's inside

1 • What is Bridgistic?	03	The plain-language explanation and the two pieces that make it work.
2 • Why it's useful	04	The problem it solves, who it's for, and the payoff.
3 • Features	05	Everything Bridgistic can do, grouped by what it's for.
4 • How it works	07	Architecture, the safety flow, scopes and keys explained simply.
5 • Setup guide	09	Six clear steps from zero to a working connection.
6 • How to use it — the best way	11	Everyday workflows, approvals, rollback, playbooks, scheduling.
7 • Security model	13	Why it's safe to run on a live site, in plain terms.
8 • Troubleshooting & FAQ	14	Quick fixes for the common snags.
9 • Glossary	15	Every technical term, defined for beginners.

► New to this? Read it in order.

Sections 1–4 explain *what* Bridgistic is and *why* it matters. Section 5 gets you set up. Section 6 shows you how to use it well. You don't need to be a developer — every term is defined in the Glossary.

SECTION 1

What is Bridgistic?

Bridgistic is a safe remote control that lets an AI assistant — Claude — make real changes to your WordPress site, without handing it the keys to everything.

You can ask Claude to write a post, fix a setting, deactivate a plugin, upload an image, clean up a database table, or run a whole multi-step task — and it actually happens on your site. The difference from doing this the usual way is that **every action is signed, limited to exactly what you allow, recorded, and reversible**.

It comes in two pieces



1. The Plugin

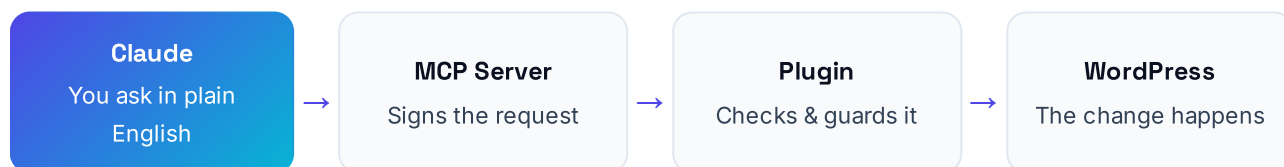
A normal WordPress plugin you install on your site. It opens a locked, guarded door that only signed requests can pass through — and it does the actual work on the site.



2. The MCP Server

A small connector that runs on your computer (or a server) and lets Claude talk to one or many sites. It signs every request and decides which site to act on.

The simple picture



► What's "MCP"?

MCP (Model Context Protocol) is the standard way AI assistants like Claude connect to outside tools. Bridgistic speaks MCP, so Claude can use it the same way it uses any other connected tool — you just talk to Claude normally.

Who it's for

- **Agencies & freelancers** running many WordPress sites who want AI to handle real work without per-site risk.
- **Product & content teams** who want to automate publishing, admin, and cleanup safely.
- **Solo site owners** who want a smart assistant that can act — but can't break anything permanently.

Why it's useful

The problem it solves

To let an AI touch your WordPress site today, you normally create an **Application Password**. That gives the AI *full administrator access* — it can do anything, to everything. If a task goes wrong, or that password leaks, your entire site is exposed. There's no undo, no record of what happened, and no way to limit it.

⚠ The risk in one line

An Application Password is all-or-nothing, permanent, silent, and irreversible. That's fine for a quick test — not for letting AI operate a client's live site.

The Bridgistic answer

Bridgistic replaces "all-or-nothing access" with a system built for trust:

	Application Password	Bridgistic
Access level	Full admin, everything	Only the scopes you grant
Risky actions	Happen instantly	Preview → approve → snapshot → run
Undo	None	One-call rollback from a snapshot
Record	No log	Every action audited
Many sites	One token each, manual	One connector, many sites
Limits	None	Rate limits + monthly quotas
If it leaks	Full site compromised	Only that key's narrow scope

What you actually get out of it

Operate at scale

Run content, admin, and maintenance across dozens of client sites from one place — without juggling dozens of full-admin tokens.

Work while you sleep

Save repeatable tasks as playbooks and schedule them to run unattended — overnight cleanups, weekly reports, recurring publishing.

Sleep at night

Destructive actions are previewed, approved, and snapshotted first. A mistake is one click back away, not a recovery nightmare.

Prove what happened

A full audit log shows who did what, when, and from where — essential for client work and accountability.

SECTION 3

Features

Everything Bridgistic does, grouped by what it's for. You don't have to use all of it — start with the basics and grow into the rest.

Security & access

Feature	What it means for you
Signed requests (HMAC)	Every request carries a tamper-proof signature. Nobody can fake or replay it. Your secret never travels over the wire.
Scoped keys	Each key can do only what you allow — e.g. "read posts" but not "delete users". Least privilege by default.
Replay protection	A time window + one-time codes (nonces) stop anyone re-sending a captured request.
IP allowlist	Optionally restrict a key to specific server IP addresses.
Encrypted secrets	Key secrets are encrypted at rest, not stored in plain text.

Safe changes

Feature	What it means for you
Dry-run preview	See exactly what a change <i>would</i> do before it happens.
Human approval queue	Risky actions can be held until you approve them in WP Admin.
Automatic snapshots	Before any destructive write, Bridgistic saves a restore point — automatically.
One-call rollback	Undo a change by restoring its snapshot in a single step.
Full audit log	Every request is recorded with who, what, when, status, and IP.

🔧 What Claude can do on your site

Area	Actions
Content	List, read, create, update, and delete posts & pages.
Media	List, upload (from a URL), and delete media library items.
Users	List, create, and update users (gated by scope).
Settings	Read and update site options — protected by an allowlist so only approved settings can change.
Plugins	List installed plugins and toggle them on/off.
Files	List, read, write, and delete files (confined to the WordPress install; executable PHP only in a sandbox).
Database	Run queries (gated by read/write scope), with snapshots on writes.
PHP	Execute PHP in a quarantined sandbox — opt-in and isolated.



Scale & operations

Feature	What it means for you
Multi-site registry	One connector manages many sites; switch by name in conversation.
Rate limits & quotas	Per-key limits per minute and per month, with billing tiers.
Per-site memory	Claude can remember useful notes about each site (e.g. "hero uses ACF fields").
Playbooks	Save a repeatable multi-step task and run it later with different inputs.
Scheduled playbooks	Run a playbook unattended on a schedule (every few minutes, daily, weekly...).



Connectivity

Bridgistic works with **Claude Desktop** and **Claude Cowork** over two transports: **studio** (the connector runs locally on your machine) and **remote HTTP** (the connector runs on a server, reachable by Cowork). Same features, your choice of where it runs.

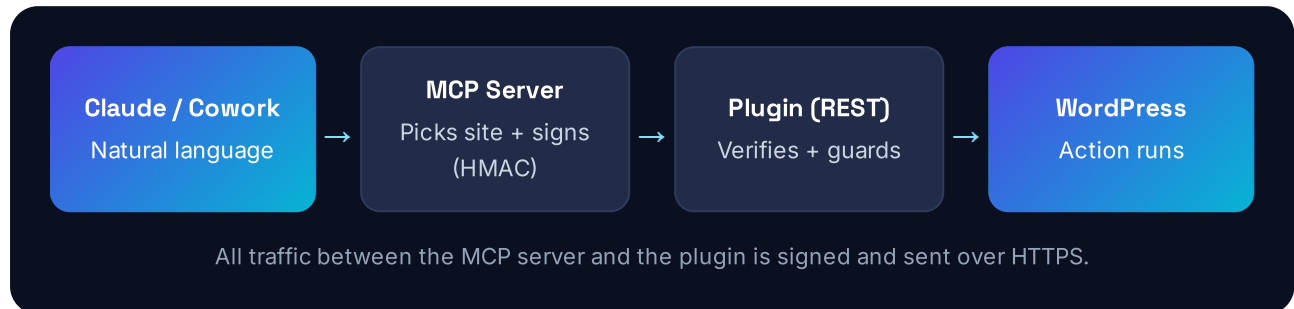
✓ The headline number

43 tools are available to Claude across all these areas — but each key only unlocks the ones its scopes allow.

SECTION 4

How it works

The architecture

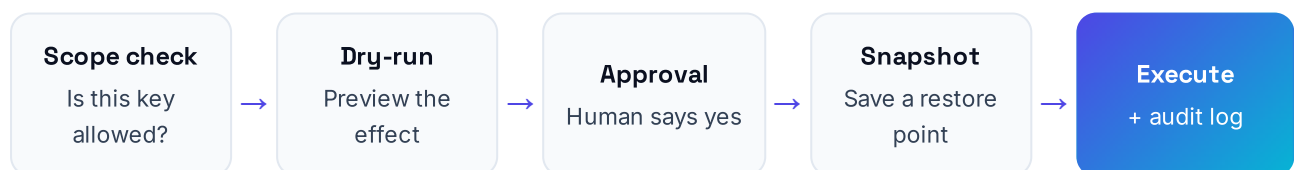


What happens in a single request

1. You ask Claude to do something (e.g. "create a draft page called Pricing").
2. The MCP server turns that into a structured request and **signs it** with your key.
3. The plugin **verifies the signature**, checks the request isn't a replay, and confirms the key's **scope** allows it.
4. If the action is destructive, the **Guard** kicks in (preview / approval / snapshot — see below).
5. The action runs, the result returns to Claude, and the whole thing is **written to the audit log**.

The safety flow (for risky actions)

This is the heart of why Bridgistic is safe. Every destructive operation passes through these gates:



✓ Why this matters

Because a snapshot is taken *before* the change, any destructive action is one rollback away from being undone. And because approval can be required, nothing risky happens without a human's "yes".

Scopes — the permission system

A **scope** is a single permission. A key holds a set of scopes, and it can do *only* what those scopes allow. Think of them like checkboxes on a key.



Ready-made presets

Preset	Good for	Includes
Read-only	First test, safe exploring	read scopes only
Content-ops	Publishing & media work	posts, media, memory
Developer	Building & maintenance	content + options + plugins + files + db
Full-trust	A site you fully control	everything, incl. php:execute

Keys — your site's credentials

A **key** is what authenticates the connection. It has two parts:

Key ID	A public identifier (like a username). Safe to store in config.
Key Secret	The private half used to sign requests. Shown only once when you create the key — copy it immediately.

⚠ The secret is shown once

When you mint a key, copy the secret straight away and store it safely (a password manager or your `.env` file). If you lose it, you can't recover it — just revoke the key and make a new one.

SECTION 5

Setup guide

Six steps from zero to a working connection. No prior experience needed — copy each command exactly.

Before you start, you'll need:

- A WordPress site (version **6.4+**, PHP **8.0+**) where you can install plugins.
- **Node.js 18+** installed on your computer (nodejs.org).
- **Claude Desktop** (or Claude Cowork).
- The two downloads: `bridgistic.zip` (plugin) and the `bridgistic-mcp-server` folder.

1 Install the plugin

In WordPress admin, go to **Plugins** → **Add New** → **Upload Plugin**, choose `bridgistic.zip`, click **Install Now**, then **Activate**.

You should now see a **Bridgistic** item in the left admin menu.

2 Mint your first key

Go to **Bridgistic** → **Connect**. Choose the **Read-only** preset for your first key (you'll widen it later), give it a name, and click **Create key**.

Copy the **Key ID** and **Key Secret** now — the secret is shown only once.

✓ Tip

Starting read-only means your very first test cannot change anything. Trust first, then widen.

3 Set up the connector (MCP server)

Open a terminal in the `bridgistic-mcp-server` folder and run:

```
# install dependencies and build
npm install
npm run build
```

This needs to be done once (and again after any update).

4 Tell the connector about your site

Single site: copy `.env.example` to `.env` and fill in your details:

```
WP_SITE_URL=https://your-site.com
BRIDGISTIC_KEY_ID=k_XXXXXXX
BRIDGISTIC_KEY_SECRET=the-secret-you-copied
```

Many sites: create a `connections.json` registry instead, and point `BRIDGISTIC_CONNECTIONS` at it. Each site gets a short alias you'll use in conversation.

5 Connect it to Claude

In Claude Desktop, open **Settings** → **Developer** → **Edit Config** and add Bridgistic to the MCP servers list:

```
{
  "mcpServers": {
    "bridgistic": {
      "command": "node",
      "args": ["/full/path/to/bridgistic-mcp-server/dist/index.js"],
      "env": {
        "WP_SITE_URL": "https://your-site.com",
        "BRIDGISTIC_KEY_ID": "k_XXXXXXX",
        "BRIDGISTIC_KEY_SECRET": "the-secret"
      }
    }
  }
}
```

Save the file and **fully restart Claude Desktop**.

6 Verify it works

Start a new chat and ask:

```
"Using Bridgistic, show me this site's WordPress version and active theme."
```

If Claude returns your site details, you're connected. 🎉 Because your key is read-only, this is completely safe.

⚠ For scheduled tasks later

WordPress's built-in cron only runs when someone visits the site. For reliable unattended schedules, add a real system cron hitting `wp-cron.php` — the **Bridgistic** → **Schedules** screen shows the exact line to use.

How to use it — the best way

► The golden rule

Start read-only. Widen trust gradually. Prove the connection works, then create a content key, then a developer key — only granting each new power once you're comfortable.

Everyday workflows

You just talk to Claude. Here's what natural requests map to:

You say...	Bridgistic does...
"List my 10 most recent posts."	Reads and returns them (read-only).
"Create a draft page titled Pricing."	Creates the draft; can preview first with dry-run.
"Deactivate the Hello Dolly plugin."	Toggles it off (with approval if required).
"Set the site tagline to 'Fast WordPress'."	Updates the option (if it's on the allowlist).
"How many API calls has my key used this month?"	Returns your usage against quota.

Previewing before committing

For anything you're unsure about, ask Claude to **dry-run** it first: "Dry-run deleting post 91 so I can see what it affects." You'll see the effect without it happening.

Approvals — keeping a human in the loop

Create keys with "**require approval**" turned on for destructive actions. When Claude attempts one, it's held in **Bridgistic** → **Approvals**. You review and click approve; only then does it run. Perfect for client sites.

Undo with snapshots

Every destructive change is snapshotted automatically. To undo: "Roll back that change using the last snapshot." Bridgistic restores the saved state in one step. You can also view snapshots under **Bridgistic** → **Snapshots**.

Playbooks — save once, reuse forever

A **playbook** is a saved sequence of steps with fill-in-the-blank inputs. Build a task once, then run it any time with new values.

Example flow:

```
# 1. Build & save
"Save these steps as a playbook called 'launch-page':
  create a page, set its template, add it to the menu."

# 2. Run any time with different inputs
"Run the 'launch-page' playbook with title = 'Summer Sale'."
```

Steps can use the result of earlier steps (e.g. the new page's ID), and the whole run still passes through every safety gate.

Scheduling — work that runs itself

Turn a playbook into a recurring job: "Run the weekly-cleanup playbook every Monday morning." Manage all schedules under **Bridgistic** → **Schedules** — pause, delete, or run-now.

✓ Best practice for schedules

Bind scheduled jobs to a key *without* "require approval" (there's no human awake at 3 a.m. to approve) — but keep that key's scopes as narrow as the task needs.

Working across many sites

With a multi-site registry, just name the site: "On *client-site*, list the draft posts." The connector routes to the right install automatically.

Do & don't

✓ Do

Start read-only · use separate keys per purpose · require approval on client sites · keep scopes minimal · test playbooks with dry-run · set up a real cron for schedules.

× Don't

Reuse one full-trust key everywhere · commit your `.env` or secret to Git · grant `php:execute` unless you need it · schedule destructive jobs without testing them first.

SECTION 7

Security model

Why it's safe to run Bridgistic on a live site — in plain terms.

Authentication	Every request is signed with HMAC-SHA256. The signature covers the method, path, time, a one-time code, and the request body. The secret itself never travels over the network.
Replay protection	Requests are valid only within a short time window and each one-time code can be used once — so a captured request can't be re-sent.
Least privilege	Keys carry an explicit list of scopes. Settings changes are allowlisted both ways. PHP can only be written to one quarantined folder where web execution is blocked.
Reversibility	Destructive actions snapshot first and can require human approval; approvals are bound to the exact action and its inputs.
Transport	HTTPS is required. Sensitive and changing values are sent in the signed request body, never exposed in the URL.
Accountability	A full audit log records every request: what, when, status, and source IP.

► Harden it further

Define `BRIDGISTIC_ENC_KEY` in your `wp-config.php` so the encryption key for stored secrets lives outside the database.

✓ The short version

Even if a key is exposed, an attacker gets only that key's narrow scope — not your whole site. And anything they could change, you can roll back.

SECTION 8

Troubleshooting & FAQ

Claude doesn't see Bridgistic after I added the config.

Fully quit and reopen Claude Desktop (not just close the window). Check the path to `dist/index.js` is absolute and correct, and that you ran `npm run build`.

"Signature" or authentication errors.

The Key ID and Secret must match a key in **Bridgistic** → **Connect**, and the secret must be copied exactly. Also make sure your server's clock is roughly correct — a very wrong time breaks the signing window.

"Scope" or "not allowed" errors.

The key doesn't have the permission for that action. Mint a key with the right preset/scopes (e.g. *content-ops* to publish), or widen the existing key.

A change is stuck "pending approval".

That key has "require approval" on. Go to **Bridgistic** → **Approvals** and approve it, then ask Claude to retry.

My scheduled playbook didn't run.

WordPress cron only fires on site traffic. Set up a real system cron against `wp-cron.php` (the Schedules screen shows the line). Also confirm the schedule is enabled and its key is still active.

Can I undo something Claude did?

Yes — if it was a destructive change, a snapshot exists. Ask Claude to roll it back, or restore it from **Bridgistic** → **Snapshots**.

Is my secret stored safely?

Key secrets are encrypted at rest in the database. For extra protection, set `BRIDGISTIC_ENC_KEY` in `wp-config.php`.

Does this work with Claude Cowork?

Yes. Run the connector in remote HTTP mode on a server Cowork can reach, and connect it the same way.

SECTION 9

Glossary

Every technical term in this guide, defined simply.

MCP	Model Context Protocol — the standard way AI assistants connect to external tools. Bridgistic speaks it.
MCP server	The small connector program that sits between Claude and your site, signing and routing requests.
Plugin	The WordPress side of Bridgistic, installed on each site, that verifies requests and does the work.
HMAC	A cryptographic signature that proves a request is genuine and unaltered, without sending the secret itself.
Key	The credential (a public Key ID + a private Key Secret) used to authenticate and sign requests.
Scope	A single permission (e.g. <code>posts:write</code>). A key can do only what its scopes allow.
Preset	A ready-made bundle of scopes (read-only, content-ops, developer, full-trust).
Dry-run	A preview that shows what an action would do, without actually doing it.
Approval	A human "yes" required before a risky action runs; managed in WP Admin.
Snapshot	An automatic restore point saved before a destructive change, so it can be undone.
Rollback	Restoring a snapshot to undo a change in one step.
Audit log	The record of every request: what, when, status, and source IP.
Nonce	A one-time code that prevents a captured request from being replayed.
Playbook	A saved, reusable sequence of steps with fill-in-the-blank inputs.
Schedule	A playbook set to run automatically on a recurrence (daily, weekly, etc.).
Sandbox	A quarantined folder — the only place executable PHP can be written, with web execution blocked.
Cron	A scheduler that runs tasks at set times. A real system cron makes WordPress schedules reliable.
Quota / Rate limit	Caps on how many requests a key can make per month / per minute.

Bridgistic · Complete User Guide · v1.0.0

By Shuvo Sarker · WordPressistic Galaxy · github.com/Shubochandrosarker/bridgistic

© 2026 WordPressistic. Released under GPL-2.0.